HashiCorp Certified: Terraform Associate（004）　目標 3a・3b・3c・3g　#04

# ワークフローとinit・validate・fmt

Write・Plan・Apply（一人・チーム・HCP Terraform）、投機的プラン、terraform init の 3 つの段階とオプション、validate が見るもの・見ないもの、fmt とスタイルの規則

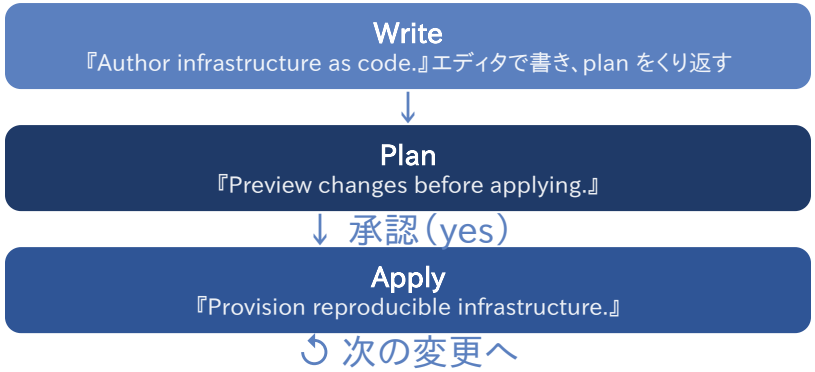
各ページの下に、根拠にした資料（HashiCorp の Exam Content List（004）、Terraform ドキュメント v1.12.x の Core Terraform Workflow Overview・Command: init・Command: validate・Command: fmt・Style Guide）を書いた。英語の原文は『』で引用した

第1章

# コアワークフロー

書く、見る、作る。そしてくり返す。

# Write → Plan → Apply 変更のたびにくり返すループ




```
$ git init my-infra && cd my-infra
$ vim main.tf
$ terraform init        # 最初に 1 回
$ terraform plan        # 書きながらくり返す
$ git commit -am '...'
$ terraform apply       # 計画を見て yes
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
$ git push origin main
```

> 一人のときは apply が出す計画が『final review』。

出典：Core Terraform Workflow Overview（Working as an Individual Practitioner）

# 一人・チーム・HCP Terraform

| 段階 | 一人 | チーム | HCP Terraform |
|---|---|---|---|
| Write | エディタ+VCS、plan をくり返す | ブランチで書く。秘密の変数が増えたら共有の CI で実行 | 変数とステートを中央に。CLI からリモートで投機的プラン |
| Plan | apply の計画を見る | PR に投機的プランを付けてレビュー。今やるか（メンテナンス時間）も判断 | PR を作ると自動でプラン。結果が PR の状態に出る |
| Apply | yes で実行し push | マージ後、最新ステートでの最終計画をもう一度確認 | 計画を承認してから実行、進み具合を共有 |

> PR で見た計画と、マージ後の計画はちがうことがある（『merge order or recent infrastructural changes』）。

出典：Core Terraform Workflow Overview（Working as a Team／The Core Workflow Enhanced by HCP Terraform）

第2章

# terraform init

まず最初に、何回でも。

# init がすること『the first command you should run』

**① バックエンドの初期化**
ルートの backend 設定を読む

↓

**② 子モジュールのインストール**
module の source から取ってくる

↓

**③ プラグインのインストール**
直接・間接に参照するプロバイダー → ロックファイルに記録

> 『always safe to run multiple times』『will never delete your existing configuration or state』

```
.
├── main.tf
├── .terraform/           # プロバイダー・モジュール（コミットしない）
└── .terraform.lock.hcl   # コミットする
```

> リソースは作らない（作るのは apply）。

出典：Command: init

# init のオプション

| オプション | 段階 | 何をするか |
| --- | --- | --- |
| -migrate-state | バックエンド | 今のステートを新しいバックエンドへコピー（-force-copy で確認なし） |
| -reconfigure | バックエンド | 今の設定を無視。ステートは移さない |
| -backend-config=… | バックエンド | 部分設定。秘密・動的な値を外から渡す |
| -backend=false | バックエンド | バックエンドを飛ばす（validate の準備など） |
| -upgrade | モジュール・プラグイン | 制約に合う最新へ。ロックファイルを無視 |
| -get=false | モジュール | 子モジュールのインストールを飛ばす |
| -lockfile=readonly | プラグイン | ロックファイルを変えず照合だけ（-upgrade と併用不可） |
| -plugin-dir=PATH | プラグイン | そのディレクトリからだけ入れる（開発中のプロバイダーなど） |

> もう一度 init：新しいモジュールだけ入る。入っているものは -upgrade まで変わらない。

出典：Command: init（Backend Initialization／Child Module Installation／Plugin Installation）

第3章

# validate と fmt

正しさと、見た目。

# terraform validate

| 見る | 見ない |
| --- | --- |
| 文法（syntactically valid） | リモートのステート |
| 中の整合性（internally consistent） | プロバイダーの API |
| 属性名・値の型 | 値がプロバイダーにとって正しいか |
| 参照先があるか | 変数の値・今のステート |

『safe to run automatically』：エディタの保存時・CI・pre-commit。

```
$ terraform init -backend=false   # バックエンドなしで準備
$ terraform validate
Success! The configuration is valid.

$ terraform validate -json
{ "valid": true, "error_count": 0,
  "warning_count": 2, "diagnostics": [ … ] }
```

警告だけなら valid は true。

特定の変数・ワークスペースで確かめるなら plan（validate をふくむ）。

出典：Command: validate／Style Guide（Code validation）

# terraform fmt とスタイル

| オプション | 動き |
| --- | --- |
| （なし） | 今のディレクトリを整えて上書き |
| -check | 書きかえず確認。整っていなければ終了コード 0 以外 |
| -diff | 差分を表示 |
| -recursive | サブディレクトリも |
| -write=false | 上書きしない |

> 『intentionally opinionated and has no customization options』。版を上げたら fmt をかけ直す。

```
resource "aws_instance" "example" {
  count = 2                    # メタ引数が最初

  ami           = "abc123"     # = をそろえる
  instance_type = "t2.micro"   # 2 スペース

  network_interface {          # 引数のあとにブロック
    # ...
  }

  lifecycle {                  # メタ引数のブロックは最後
    create_before_destroy = true
  }
}
```

> コミット前に fmt と validate。組織の規則は TFLint。

出典：Command: fmt／Style Guide（Code style／Code formatting）

# 本番での問われ方

| ねらい | 見るところ |
| --- | --- |
| ワークフロー | Write → Plan → Apply の順、チームではブランチ・PR・最終計画の再確認 |
| HCP Terraform | PR で自動の投機的プラン、変数とステートを中央に |
| init | 最初に実行、何回でも安全、3 つの段階、リソースは作らない |
| init のオプション | -migrate-state と -reconfigure、-upgrade、-backend-config、-lockfile=readonly |
| validate | 文法と型は見る、API・ステート・値の妥当性は見ない、-backend=false で準備 |
| fmt | -check は CI、-recursive、設定項目なし、2 スペースと = のそろえ |

出典：Exam Content List（004）3a・3b・3c・3g／本資料の問題演習（全 30 問）